

Unité 1 : Notion d'algorithme

Contrôle de presrequis :

1. Enumerer les étapes de résolution d'un problème.

Compétences visées :

- Identifier les éléments d'un algorithme ;.
- Construire un organigramme ;

Situation probleme :

De nos jours, il n'est pas rare d'utiliser un navigateur GPS pour obtenir un itinéraire (but de l'algorithme). On entre alors le point de départ et le point d'arrivée (données d'entrée – 1 ère phase). Une série d'instructions (traitement des données – 2 ème phase) fournit en sortie une ligne brisée (résultat – 3 ème phase) qui symbolise le chemin à parcourir pour joindre ces deux points.

Mais comment écrire un algorithme pour qu'il soit universellement compréhensible ? Un algorithme peut être soit écrit sous forme littérale (langage algorithmique), soit représenté graphiquement (algorigramme).

Guide d'interactivité :

- **Qu'est-ce qu'un algorithme ?**

Réponse : un algorithme une suite finie et ordonnée d'opérations élémentaires donc l'exécution pas à pas permet de résolution un problème;

- **Quels sont les caractéristiques d'un algorithme ?**

Réponse : Un algorithme doit être simple, non ambiguë, compréhensible, ordonné, durable, lisible et finitude.

- **Donner sa structure générale.**

Réponse : un algorithme a 3 partie : l'entête, la partie déclarative et le corps.

- **Identifier les éléments dans un algorithme.**

Un algorithme comporte plusieurs éléments : les variables et constantes, les fonctions et procédures, les instructions(écriture, lecture, affectation, incrémentation/décrémentation, les opérateurs (arithmétique, logique, booléen, ...), etc.

1- DÉFINITION

Un algorithme est: une suite finie et ordonnée d'opérations élémentaires dont l'exécution pas à pas permet de résoudre un problème;

Un algorithme est: une méthode de résolution systématique d'un problème pouvant être réalisé de façon mécanique;

L'algorithmique : est la science qui étudie les algorithmes.

Un programme : est la réalisation d'un algorithme dans un langage donné (proche de celle de la machine);

Un langage de programmation : est un langage destiné à décrire un ensemble d'actions consécutives qu'un ordinateur doit exécuter.

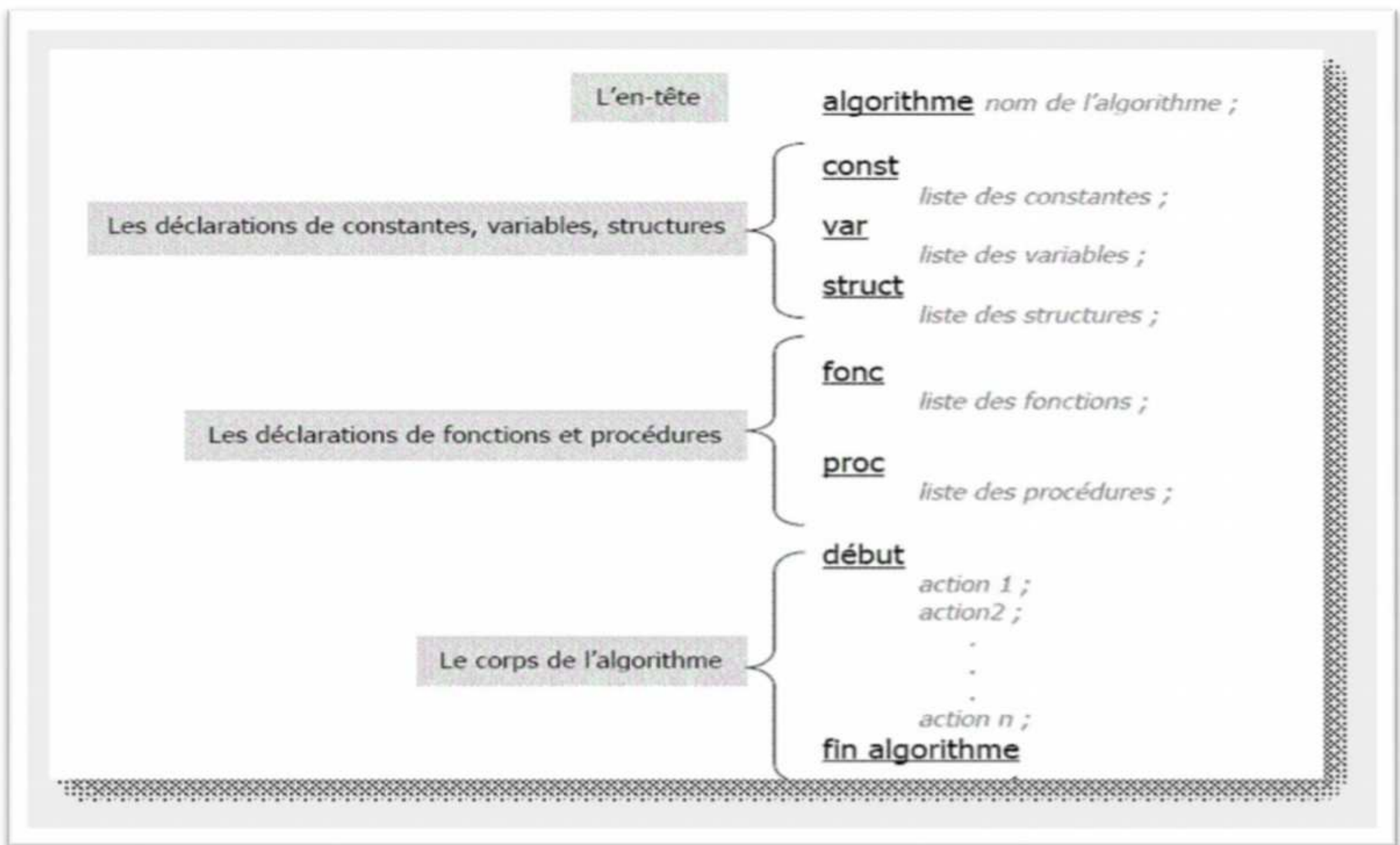
2- Caractéristiques d'un algorithme

Un algorithme doit être: simple, non ambiguë, compréhensible, ordonné, durable, lisible et finitude :

- **La finité:** la description des procédés doit être de longueur finie.
- **Le déterminisme:** un algorithme est dit déterministe si les étapes d'exécution sont bien fixées et ne conduisent à des choix aléatoires.
- **La terminaison (finitude):** l'algorithme doit produire la sortie souhaitée avec un nombre fini d'étapes.
- **La généralité:** l'algorithme s'applique à tous les problèmes d'une même classe.
- **La correction d'un algorithme** signifie qu'il doit répondre au problème pour lequel il a été conçu.
- **La clarté (lisibilité)** d'un algorithme implique que le concepteur s'assure qu'il est facile à comprendre et à interpréter. Chaque opération doit être définie d'une manière précise.
- **La documentation** d'un algorithme consiste à l'insertion des connecteurs qui facilitent la compréhension du programme.

NB : Un algorithme est dit efficace lorsque les opérations sont suffisamment simples et qu'elle s'exécute le plus rapidement possible.

3- La structure générale d'un algorithme



Tout comme le corps humain, un algorithme a trois parties:

- **L'entête:** Permet tout simplement d'identifier l'algorithme. L'algorithme peut prendre n'importe quel nom.
- **La partie déclarative:** Permet de déclarer tous les objets à utiliser dans le corps de l'algorithme: variable, constante, structure, fonction, procédure.
- **Le corps de l'algorithme:** Compris entre les mots "début" et "fin" contient les instructions, les délimiteurs, les opérations de traitements et les commentaires.

4- VARIABLES ET CONSTANTES

Une constante: est un objet ayant une valeur fixe tout au long d'un algorithme. En d'autres termes sa valeur ne change pas. Elle est caractérisée par deux éléments : son nom (identificateur) et sa valeur.

Une variable: est un objet pouvant prendre différentes valeurs tout au long d'un algorithme. Elle est caractérisée par trois éléments: son identificateur (nom), son type et son contenu.

Il existe **5 types de variable existent:** Entier (integer), réel (real), booléen (boolean), caractère (character), chaîne de caractère (string).

5- INSTRUCTIONS

Les instructions sont les ordres de traitement respectant les actions simple dans l'exécution d'un algorithme.

Il existe plusieurs instructions :

- **L'instruction d'affectation:** Elle consiste à attribuer une valeur à une variable. On utilise le symbole $\leftarrow$ qui signifie égal (=) en mathématiques.
Exemple: $a \leftarrow 2$ signifie que a prend la valeur 2 ou 2 est affecté à a ou $a=2$.
- **L'instruction de sortie :** Elle consiste à écrire une donnée sur un périphérique de sortie tel que l'écran ou l'imprimante,... Elle se réduit au verbes: afficher (); écrire (); writeln (); Sa syntaxe est: Ecrire ("entrer la valeur de a);
- **L'instruction d'entrée:** Une entrée consiste à introduire une donnée à partir des sources d'entrée (clavier, souris, crayon optique, ...). Elle permet d'affecter à un objet en mémoire une valeur de même type que l'objet.
- Elle s'utilise par les mots "lire ()", "saisir ()", "Readln ()". Syntaxe : Saisir (a).
- **Les instructions d'incrément/ de décrémentation:** Elles se rencontrent dans les boucles. L'incrément peut être assimilé à un compteur qui à chaque cycle augmente d'une unité (1); La décrémentation est la diminution d'une unité à chaque cycle. Pour ce faire on utilise les variables du compteur (i ou j).

Syntaxe: $i \leftarrow i+1$; $j \leftarrow j-1$

NB : Les trois premières instructions sont dites élémentaires.

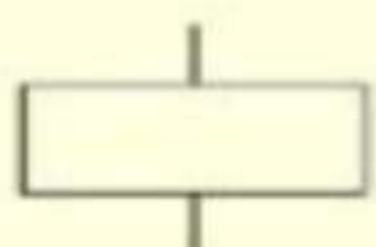
6- LES OPÉRATEURS

On distingue plusieurs types d'opérateurs:

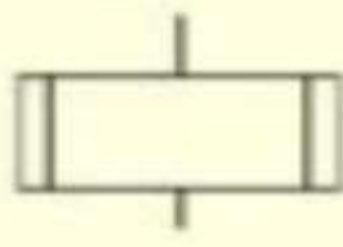
- **Les opérateurs arithmétiques:**
 - **Les opérateurs unaires ou monodiques** (appliqué à un seul opérande) : -27;
 - **Les opérateurs binaires ou diadique** (lient deux opérandes) : **12+5**;
- **Les opérateurs logiques ou booléens:** ET, OU, NON.
- **Les opérateurs relationnels:**
 - Inférieur (<, <=); supérieur (>, >=);
 - égalité (=); différence (<>).

7- Organigramme

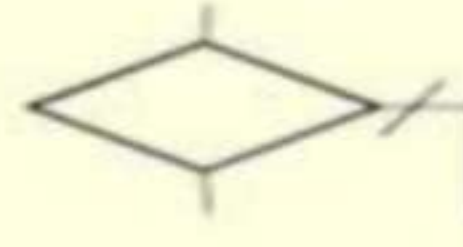
Représentation graphique de l'algorithme. Pour le construire, on utilise des symboles normalisés.



Symbole général
Opération ou groupe d'opérations sur des données, des instructions,....



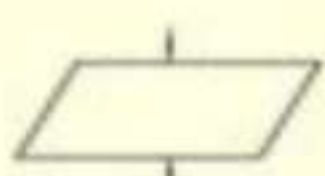
Sous-programme
Portion de programme considérée comme une simple opération.



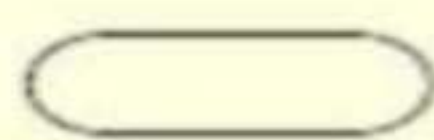
Branchement
Exploitation de conditions variables impliquant un choix parmi plusieurs.



Renvoi



Entrée-Sortie



Début, fin , interruption

Le sens général des lignes de liaison doit être :

- De haut en bas
- De gauche à droite

Activité d'intégration

Algorithme Plusgrand

Variable a, b, pg : réel ;

Début

Ecrire ('entrer les deux valeurs') ;

Lire (a, b) ;

Si a>b alors

Pg ← a ;

Sinon

Pg ← b ;

FinSi

Ecrire ('le plus grand nombre est :', pg) ;

Fin

a) Que fait cet algorithme ?

b) Combien de variables utilise cet algorithme ? donner leur type.

c) Quelles sont les différentes instructions présentes dans cet algorithme ?

Donner leurs types.

d) Expliquer les éléments : « Ecrire ('entrer les deux nombres'), pg ← a.

Solution

a) Cet algorithme détermine le plus grand entre deux nombres donnés.

b) Les variables utilisées par cet algorithme sont tous des réels : a, b, pg.

c) Il y a 3 instructions dans cet algorithme : lecture, écriture et affectation.
Il y a un seul opérateur logique : supérieur (>).

d) Les instructions :

- **Ecrire ('entrer les deux nombres')** signifie que l'utilisateur doit saisir deux nombres au clavier.
- **Pg ← a** signifie que **pg** prend la valeur a

Réinvestissement

Algorithme Dagaarii

Variable x, y, i : entier

Début

y ← 1 ;

Ecrire ('entrer la valeur de x') ;

Lire (x) ;

Ecrire ('entrer la valeur de n') ;

Lire (n) ;

Pour i ← 1 à n faire

y ← y*x ;

i ← i+1 ;

FinPour

Ecrire (y) ;

Fin

Dans cet algorithme :

a) Donner le nombre d'instruction par type

b) Identifier les différentes variables et leur type.

c) Quels sont les opérateurs présents ici ?

d) Quelle différence y a-t-il entre les instructions : Ecrire('entrer la valeur de n') et Ecrire (y)

Unité 2 : Structures algorithmiques (séquentielle et contrôle)

Contrôle de presrequis :

- Connaître la structure générale d'un algorithme ;
- Identifier les instructions.

Compétences visées :

- Identifier les structures (contrôle et séquentielle);
- Construire un organigramme ;
- Connaître quelques opérations mathématiques de base.

Situation problème

Écrire un algorithme permettant de résoudre une équation à coefficients réels de la forme $ax+b = 0$ (a et b seront entrés au clavier).

Guide d'interactivité

- **Lister les différentes structure de contrôle.**

Réponse attendue : structures alternatives ou conditionnelles, les structures itératives ou boucles.

- **Quelle structure convient-il ici ?**

Réponse attendue : structure alternative car il ya une condition sur le coefficiet a ($a=0$ ou $a\neq 0$)

- **Donner sa syntaxe**

Si (condition) alors
 (actions)
sinon (actions)

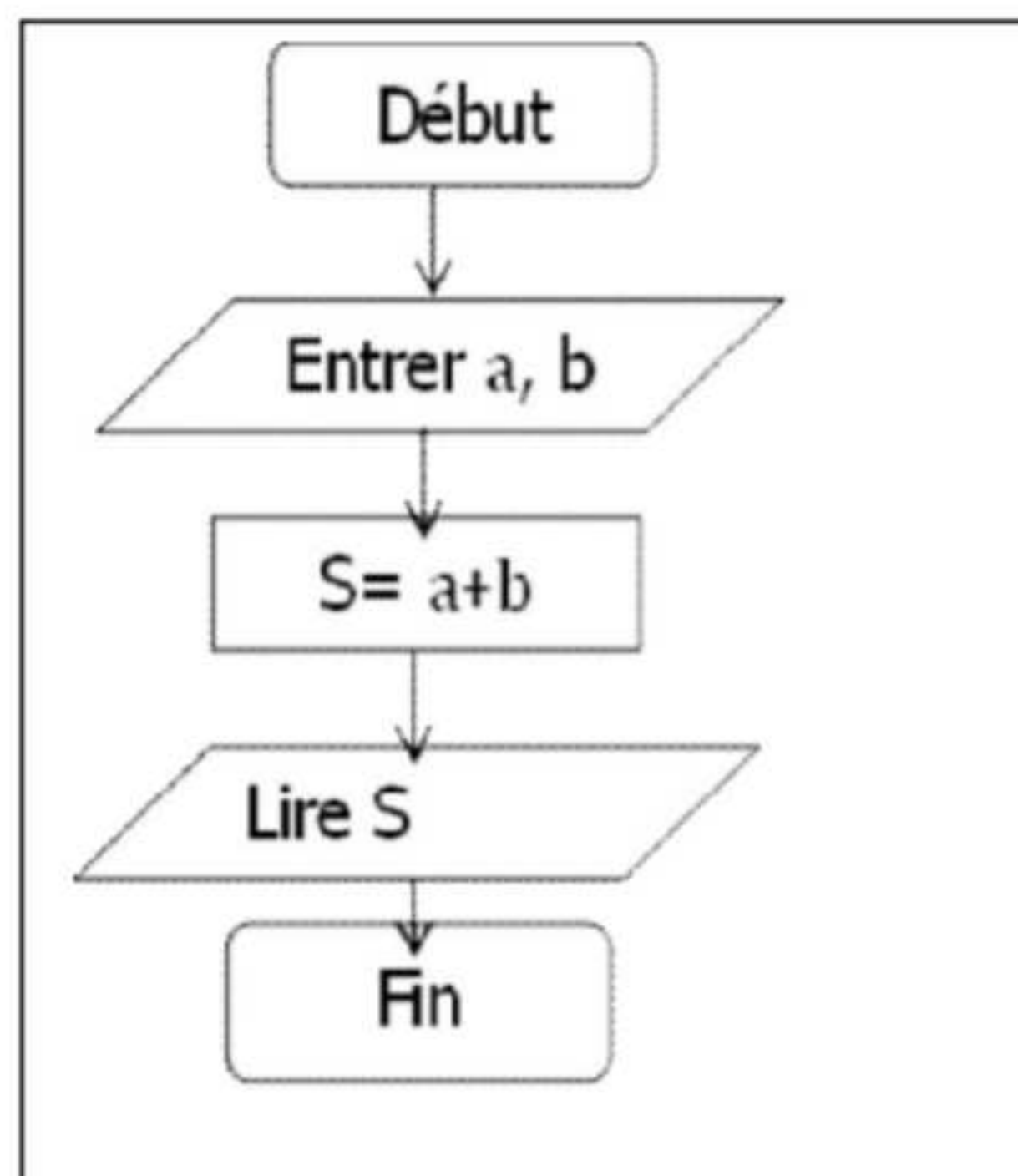
On distingue 3 structures algorithmiques de base: linéaire ou séquentielle, conditionnelle, itérative ou boucle.

1- STRUCTURE LINÉAIRE OU SÉQUENTIELLE

Dans cette structure, les actions se suivent, la fin d'une action déclenche le début d'une autre sans interruption. Une action se termine par un point virgule (;).

Exemple d'algorithme

- Algorithme somme
 - Variable a, b: réel
 - s: réel
- Début
 - Écrire ("entrer le premier nombre");
 - Lire (a);
 - Écrire ("entrer le deuxième nombre");
 - Lire (b);
 - $s \leftarrow a+b$;
 - Écrire (s);
- Fin



2- STRUCTURES CONDITIONNELLES

Une structure conditionnelle est dite à forme alternative lorsque le traitement dépend d'une condition à deux états:

- Si la condition est évaluée à vrai, le premier traitement est exécuter;
- Si la condition est évaluée à faux, le 2^{ème} traitement est exécuté.

Une structure conditionnelle est dite à choix lorsque le traitement dépend de la valeur que peut prendre le sélecteur. Ce sélecteur doit être du type scalaire (entier ou caractère);

Une structure conditionnelle est dite généralisée lorsqu'elle permet de résoudre les problèmes comportant plus de deux traitements en fonction des conditions. L'exécution d'un traitement entraîne automatiquement la non exécution des autres (alternative multiple).

a) Structure conditionnelle à forme alternative

On distingue deux structures à forme alternative:

- L'alternative réduite:

Elle ne s'exécute que si la condition est satisfaisante.

syntaxe : si (condition) alors (instruction)

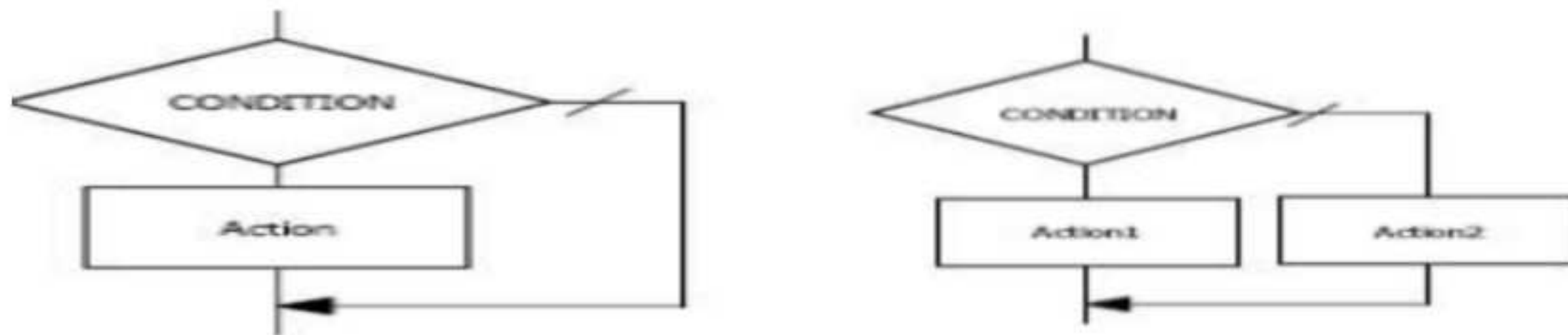
Exemple : Si $x \neq 0$ alors $s \leftarrow \emptyset$

- **L'alternative complète**

Dans ce cas, une instruction est prévue pour chaque valeur logique de condition.

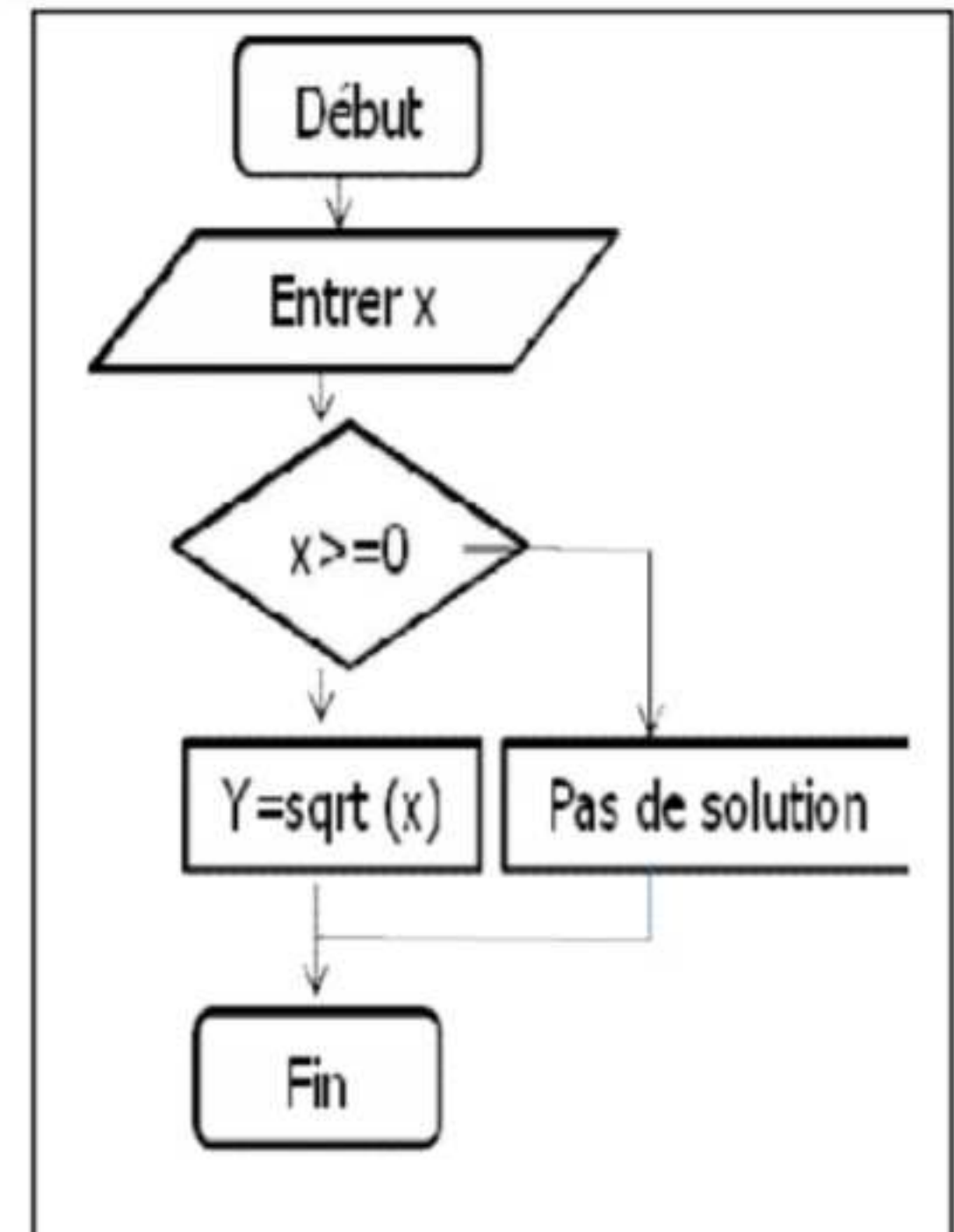
syntaxe : si (condition) alors (instruction) sinon (instruction)

Exemple: Si $x \neq 0$ alors $s \leftarrow b$ sinon $s \leftarrow \emptyset$



Exercice : racine carré d'un nombre

- Algorithme Squart
 - Var x, y: réel
- Début
 - Ecrire ("enter un nombre");
 - Lire(x);
 - Si $x \geq 0$;
 - $y \leftarrow \text{sqrt}(x)$;
 - Ecrire (y);
 - Sinon
 - Ecrire ("pas de racine réelle");
 - Finsi
- Fin



b) L'alternative multiple

La conditionnelle Si ... Alors ... Sinon ... FinSi peut être complétée avec des clauses

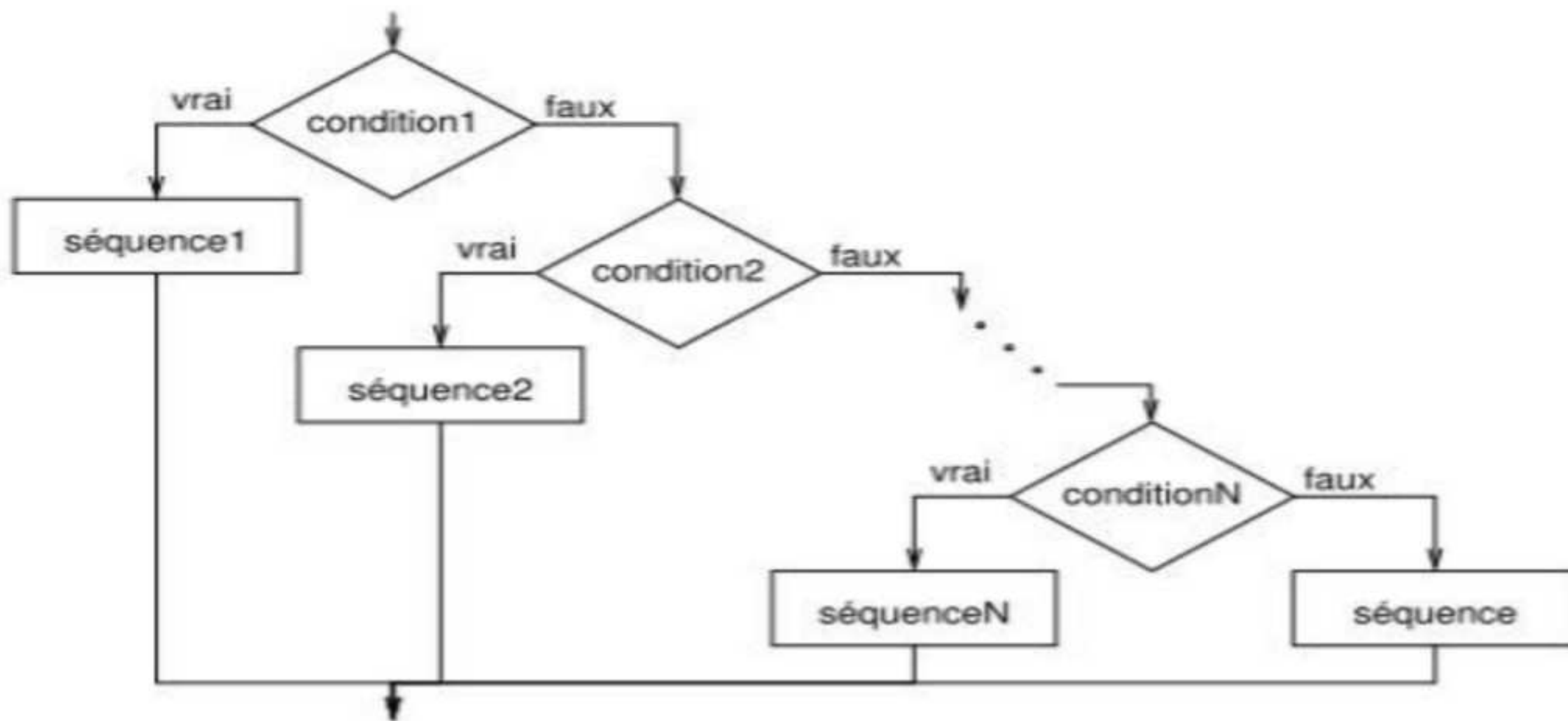
Sa syntaxe est la suivante :

```

Si condition1 Alors
    séquence1
SinonSi condition2 Alors
    séquence2
....
SinonSi conditionN Alors
    séquenceN
Sinon { Expliciter la condition ! }
    séquence
FinSi
  
```

Évaluation : Les conditions sont évaluées dans l'ordre d'apparition. Dès qu'une condition est vraie, la séquence associée est exécutée. L'instruction suivante à exécuter sera alors celle qui suit le FinSi. Si aucune condition n'est vérifiée, alors la séquence associée au Sinon, si elle existe, est exécutée.

Organigramme :



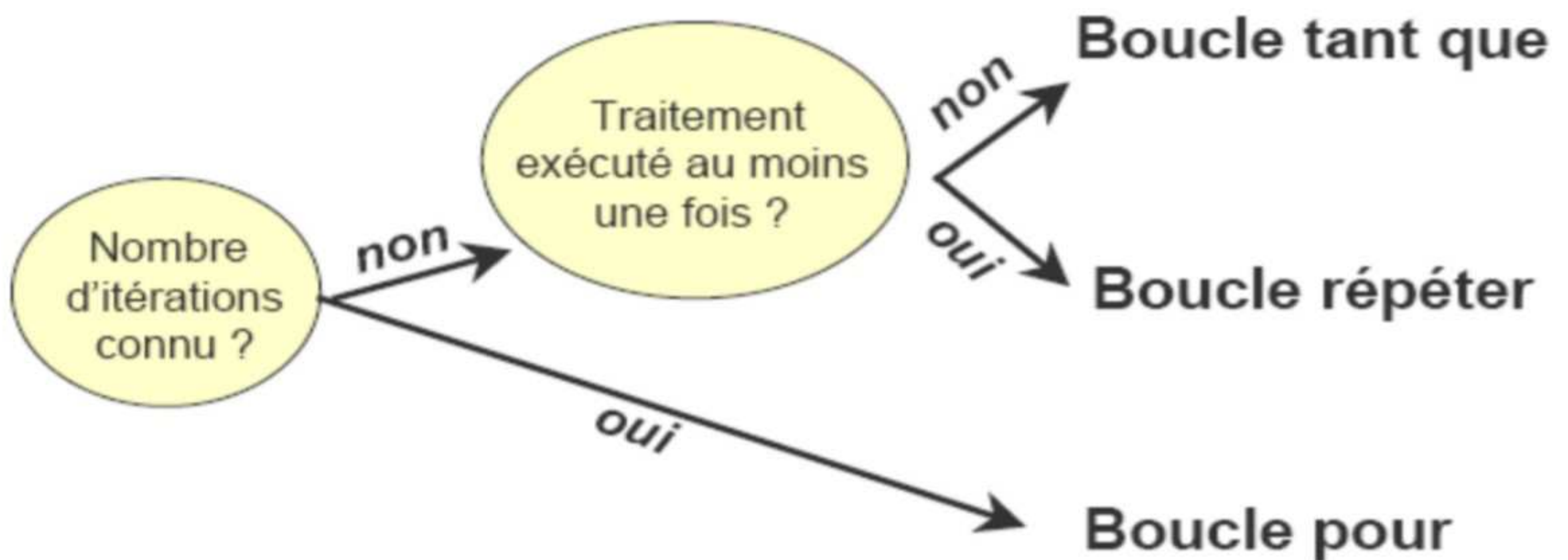
2- STRUCTURES ITÉRATIVES OU BOUCLES

a) Définition

Une itération ou boucle est une séquence d'instructions destinée à s'exécuter plusieurs fois.

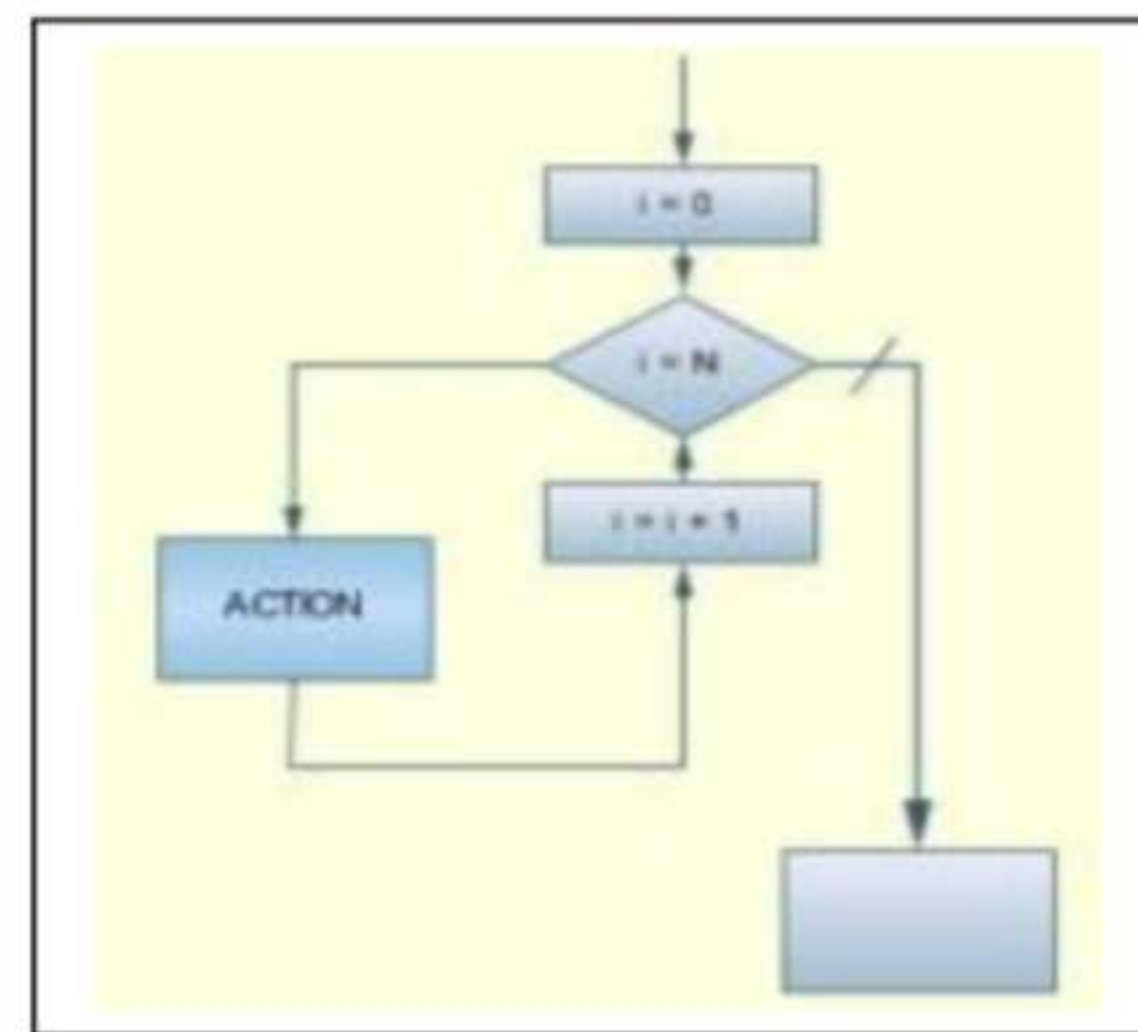
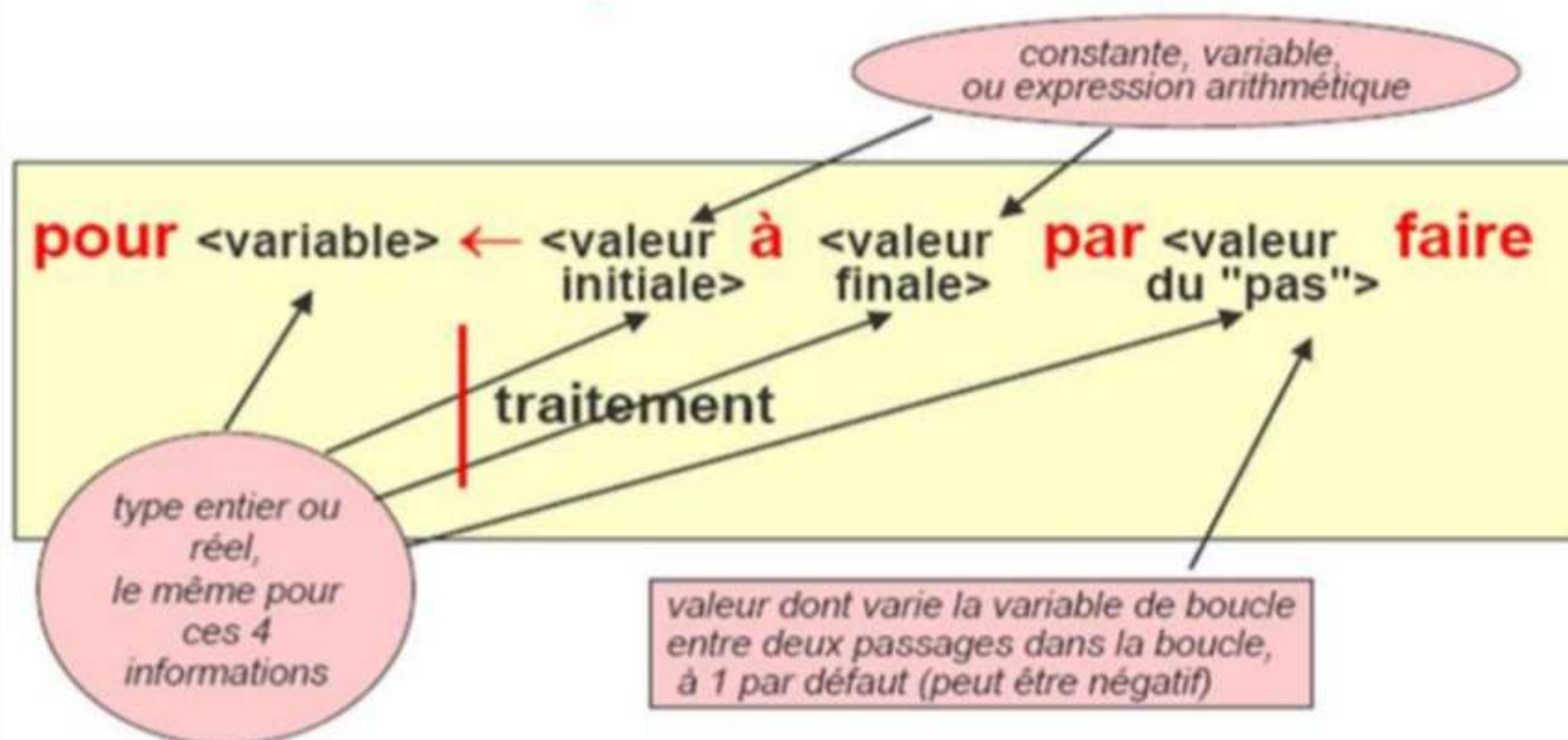
Un cycle est un tour de la boucle.

On distingue deux types d'itérations: itérative complète (pour) et itérations à condition d'arrêt (tant que, répéter).



➤ Itération complète : la boucle pour

- Permet de passer n fois dans une séquence d'instructions.
- Utilise le compteur;
- Le compteur incrémente la variable à chaque pas, vérifie que cette variable ne dépasse pas la borne supérieure.
- S'utilise lorsque le nombre d'itération à exécuter est bien connu.
- Sa syntaxe est Pour (compteur) ← initial au final faire.



Exemple calculer la puissance d'un nombre

Algorithme puissance

Var n, i : entier; x, p: réel

Début

P ← 1;

Pour i ← 1 à n faire

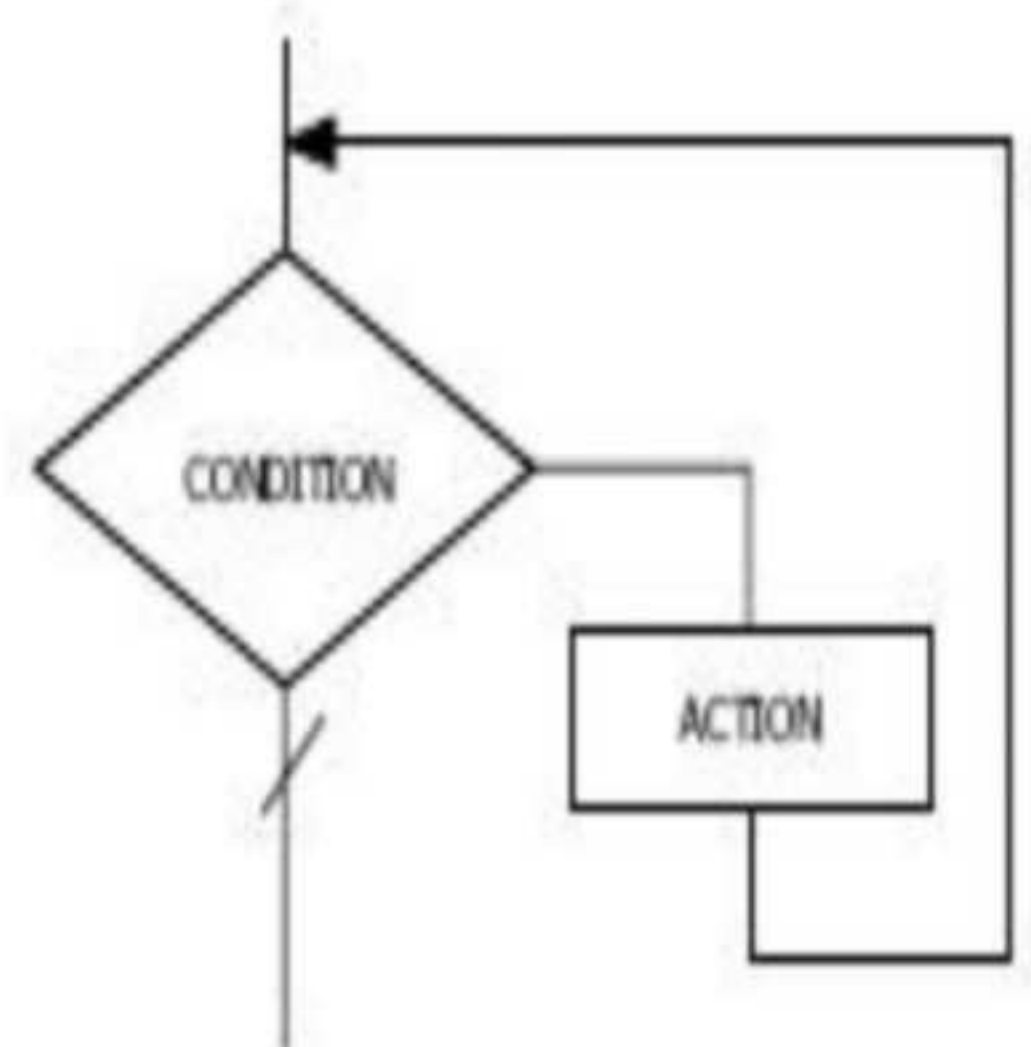
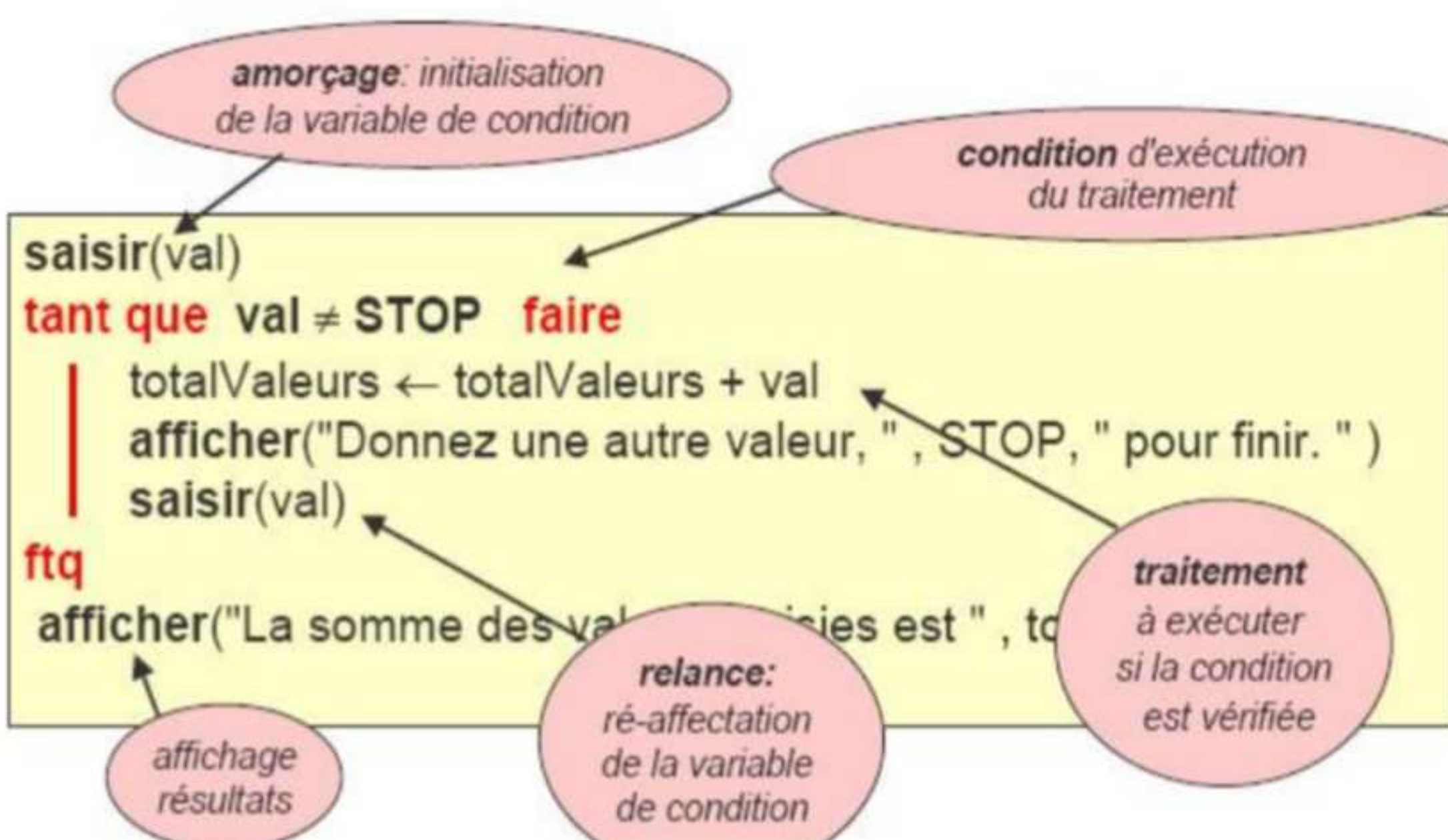
p ← p * x

finPour

Fin

➤ Itération à condition d'arrêt : La boucle Tant que

- Est une structure dans laquelle la condition est d'abord testée et le traitement exécuté si la condition est remplie.
- S'utilise lorsqu'aucun traitement ne s'exécute avant la condition (le test) et le nombre d'itération n'est pas connu.
- Doit initialiser un compteur (amorçage), incrémenter le compteur à chaque pas (relance), vérifie que le compteur ne dépasse pas la borne supérieures (test de boucle).
- Sa syntaxe est: tant que <condition> faire (instruction)



Exercice

Vous êtes dans une station de service de transport public à attendre le moyen de transport désiré. Le dernier arrive, le convoyeur procède à faire monter les voyageurs avec la condition qu'aucun ne doit voyager debout.

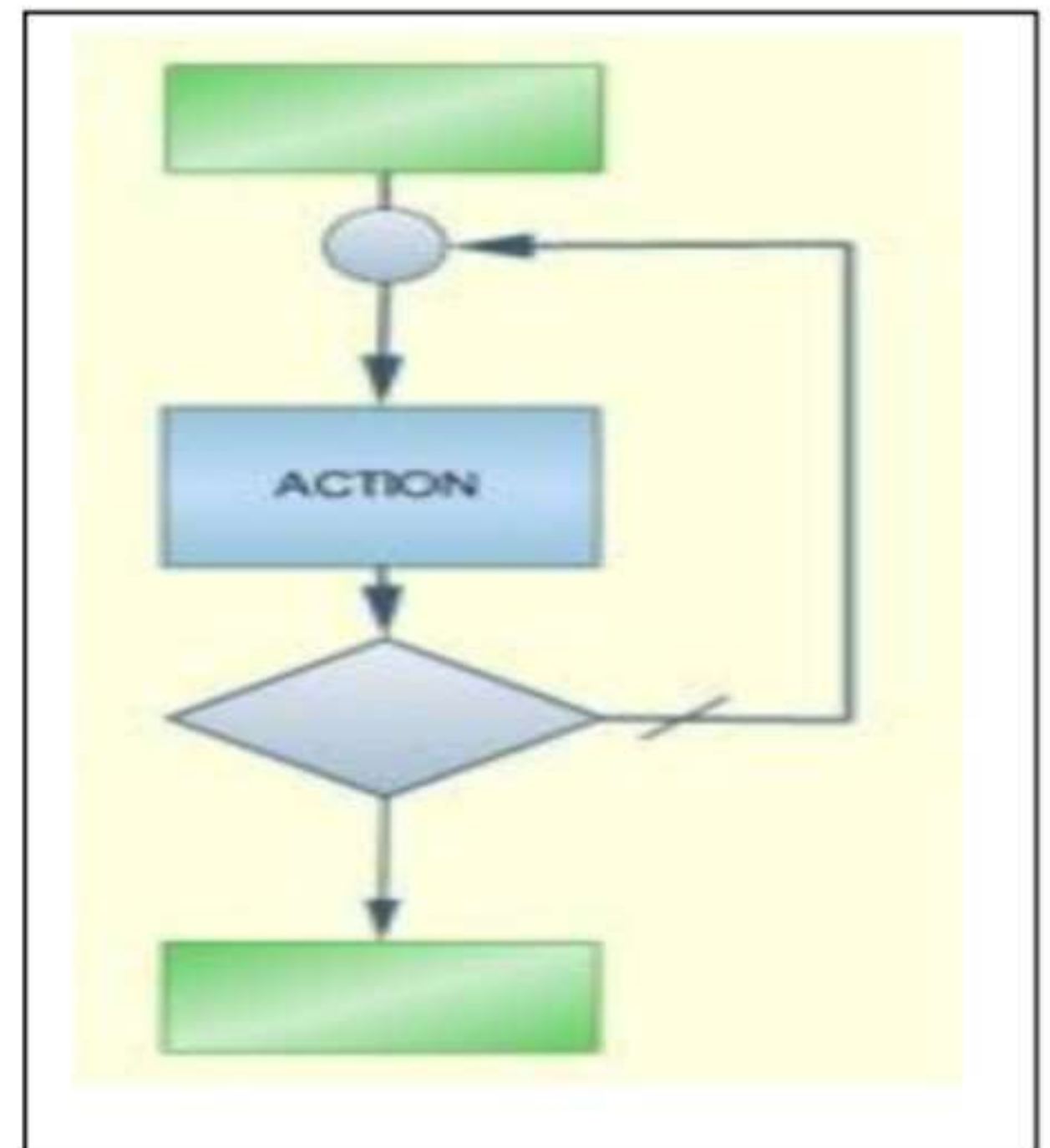
Solution

- Le convoyeur avertit le chauffeur d'ouvrir les portes s'il reste des places vacantes. Il commence l'action itérative suivante:
 - Tant qu'il ya encore de la place, faire monter un voyageur.
- La boucle peut être formulée ainsi:
- (arrêter le bus)
- tant que (il ya de place vacante) faire
 - monter le passager
- Fin tanque

➤ Itération à condition d'arrêt : La boucle Répéter

- Est une structure dans laquelle le traitement est exécuté une fois avant le test et le nombre d'itération n'est pas connu.
- Syntaxe: Répéter (instruction) jusqu'à (condition)
- Exemple: un algorithme qui calcule la moyenne des notes.

- Algorithme Moyenne
- Var s: réel; note: entier
- Début
- $s \leftarrow 0$;
- $i \leftarrow 1$;
- Répéter
- Ecrire ("donner la note n°:", i);
- Lire (note);
- $s \leftarrow s + \text{note}$;
- $i \leftarrow i + 1$;
- Jusqu'à $i \geq 10$
- Ecrire ("la moyenne des 10 notes est:", $s/10$);
- Fin



ACTIVITE D'INTEGRATION

On souhaite écrire un algorithme qui demande un nombre de départ, et qui ensuite écrit la table de multiplication de ce nombre, présentée comme suit (cas où l'utilisateur entre le nombre 7) : Table de 7 : $7 \times 1 = 7$; $7 \times 2 = 14$; $7 \times 3 = 21$; ; $7 \times 12 = 84$ **TAF**

- 1) Identifier la structure algorithmique correspondante
- 2) Ecrire l'algorithme complète resolvant ce problème.

Solution

- 1) Structure itérative (plusieurs répétitions de la même action). Comme le nombre de répétition est connu d'avance, on utilisera l'itération complète (boucle Pour)

- 2) Algorithme

Algorithme TableMultiplication

Variables n, i : Entier

Debut

Ecrire ("Entrez un nombre : ") ;

Lire (n) ;

Ecrire("La table de multiplication de ce nombre est : ") ;

Pour $i \leftarrow 1$ à 10

Ecrire (n , " x ", i , " = ", $n*i$) ;

$i \leftarrow i+1$

Fin

Reinvestissement

Exercice 1. Calcul du $n^{\text{ième}}$ nombre d'une suite

Écrire un algorithme permettant de calculer la $n^{\text{ième}}$ valeur d'une suite de la forme $U_n = aU_{n-1} + b$, $U_0 = c$ (a , b et c sont des entiers naturels entrés au clavier).

Exercice 2. Calcul d'antécédent par une fonction affine

Écrire un algorithme qui calcule, s'il existe, l'antécédent d'un nombre y par une fonction affine $f(x) = ax + b$ (a et b donnés).

Exercice 3. Tracé de courbe d'un polynôme de degré 2

Écrire un algorithme permettant de tracer la courbe de la fonction polynôme de degré 2 $f(x) = ax^2 + bx + c$ (a , b et c donnés).

Exercice 4. Lancer de pièces

Écrire un algorithme permettant de simuler n lancers de pièces, n donné, et d'afficher la fréquence de l'événement « la pièce tombe sur Pile ».